

IEEE POSIX thread (*pthread*) library

for



As we know...

- ◆ Every process must have at least *one* thread of execution
- ◆ Function “*main*” forms this default thread


```
int main()
{
    cout << "Hello world!\n";
    return 0;
}
```



```
int main()
{
```

```
    printHelloWorld();
```

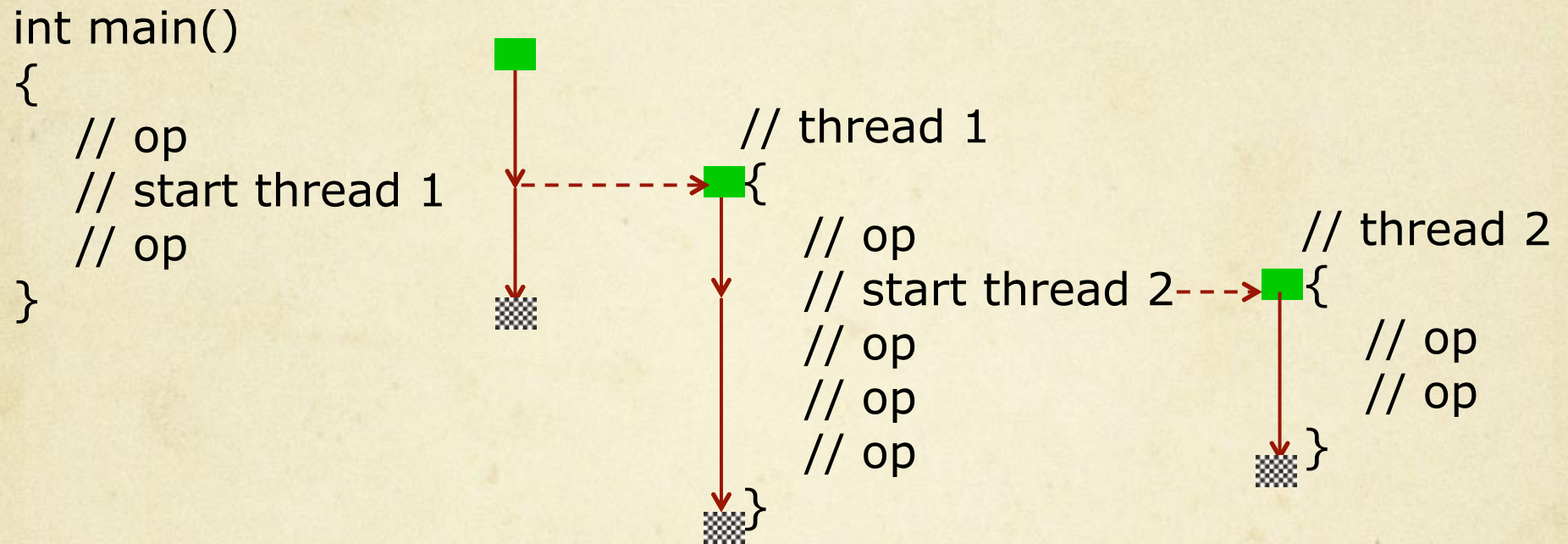
```
    // other ops
    return 0;
```

```
}
```

```
void printHelloWorld()
```

```
{
    cout << "Hello world!\n";
}
```

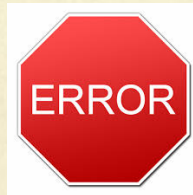




Pthread library becomes useful to do this!

Important notes

- ◆ Include file: *pthread.h*
- ◆ Linker flag: *-lpthread*
 - ◆ If program file is test_pthread.cpp, command would be:
g++ test_pthread.cpp -lpthread
- ◆ Undefined reference



We need to...

- ◆ Setup *sequence of instructions*
 - ◆ Create function with desired sequence of instructions

`void* func(void*)`

- ◆ `void*` can point to any type of data!
- ◆ *Create* new thread to *execute* sequence of instructions

Thread creation

placeholder for thread ID

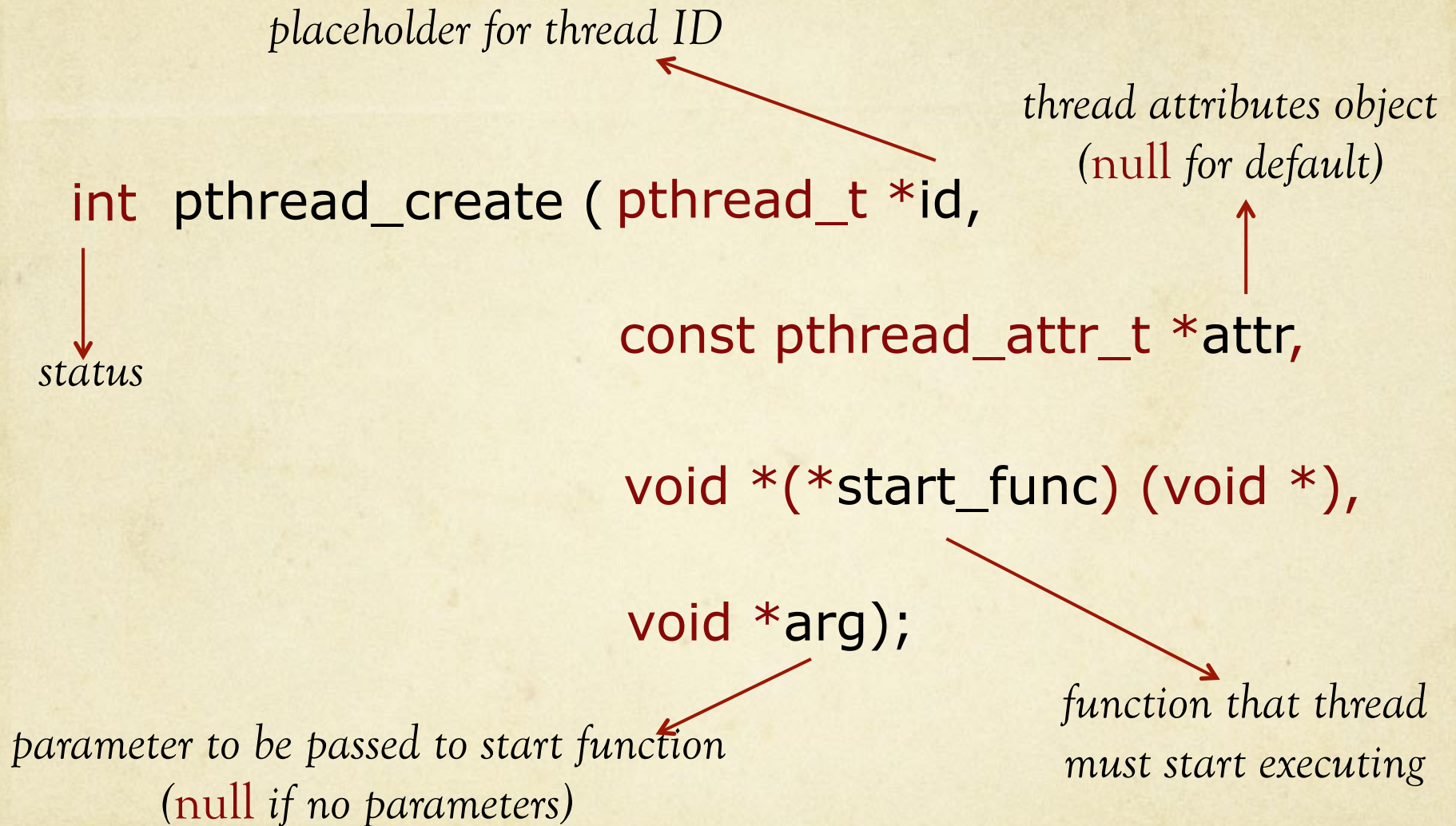
*thread attributes object
(**null** for default)*

```
int pthread_create ( pthread_t *id,  
                    const pthread_attr_t *attr,  
                    void *(*start_func) (void *),  
                    void *arg);
```

status

*parameter to be passed to start function
(**null** if no parameters)*

*function that thread
must start executing*



Why no hello? ☹️

- ◆ By default, *scheduling* of threads is up to system
 - ◆ No guarantee *when* and in *what order* threads execute
- ◆ Function *main* implicitly calls *exit* function when it's done
 - ◆ Terminates *process*, i.e., all threads – even if they're not done
- ◆ Use different function to terminate only *calling thread*

void pthread_exit (**void** *retval)

placeholder for return value from terminating thread ↗

Parameters & return values

- ◆ Any parameter to be passed must be converted to void*

int arg;

void to_pass = (void*) &arg;*

- ◆ Similarly, return values must also be converted to void*

char retval;

void to_return = (void*) retval;*

Parameters & return values

- ◆ What if there is more than one parameter to pass?
 - ◆ Consolidate parameters into single structure
 - ◆ Convert & pass pointer to structure as parameter

```
struct my_args  
{  
    int arg_1;  
    char arg_2;  
};
```

```
my_args a;  
a.arg_1 = 10;  
a.arg_2 = 'Y';
```

```
void* to_pass = (void*) &a;
```